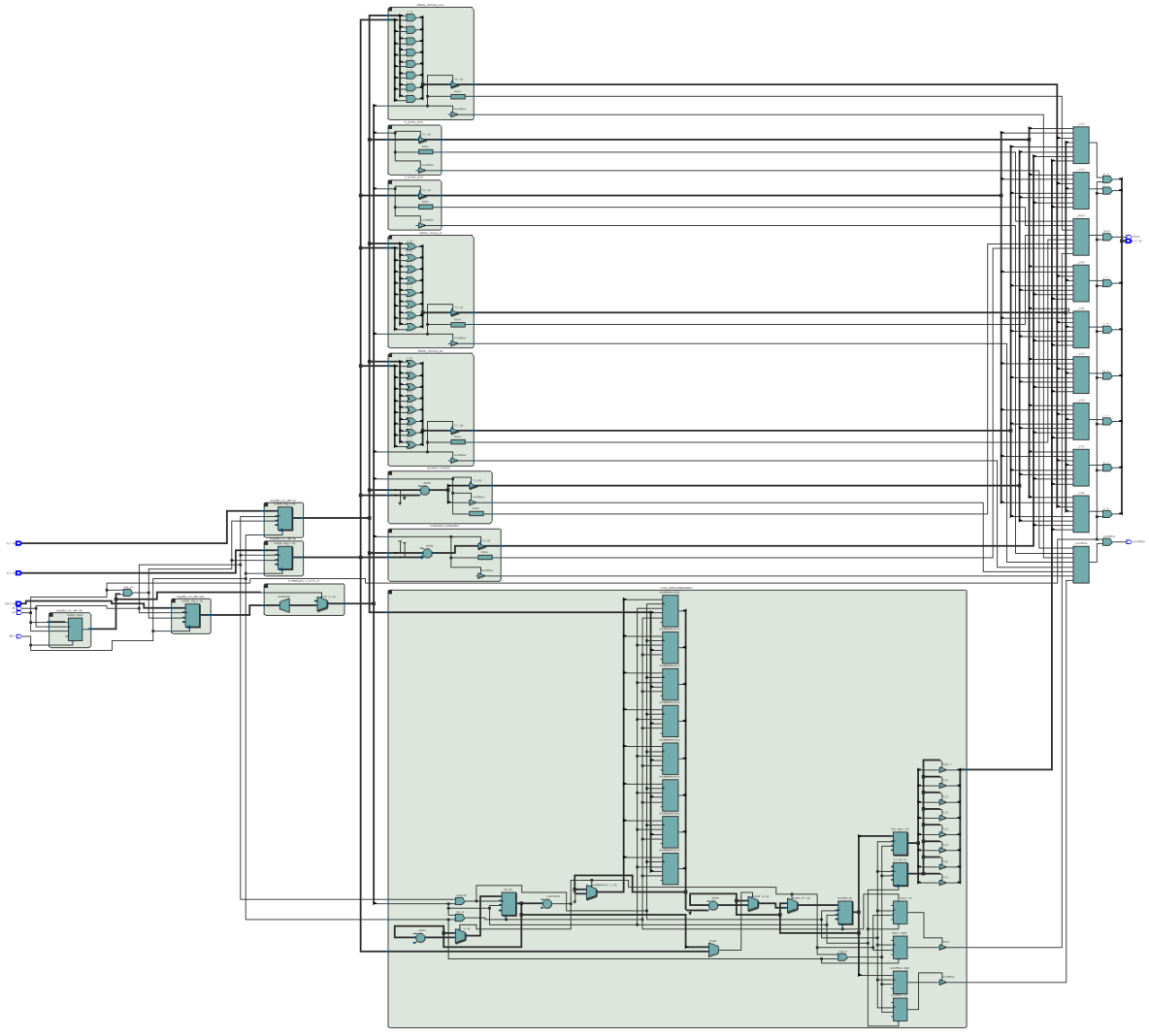
ECE 204 Design Project

8 Bit ALU

Nolan Kessler, Maxwell Fabian, John O'Connor

June 7, 2026



1 8 Bit ALU Functional Description

This 8 bit ALU accepts two 8 bit operands A and B, along with a 3 bit opcode (OP). The state of the ALU is controlled by an active high enable pin (EN), and can be reset using the active low reset pin (RST_N). As this ALU supports sequential operations which may take more than one clock cycle, the CLK input must be connected to a clock signal. The ALU provides one 8 bit result output (Y) an overflow flag (OVER) and a done flag (DONE). A table of the ALU's supported operations is given in table 1. A diagram of the inputs and outputs is shown in figure 1. A descriton of the function of each bus/pin is given in table 2.

Operation	**Opcode**	**Clock Cycles**	**Overflow Condition**	**Description**
A pass	0x0	1	N/A	Passes A to Y unmodified
B pass	0x1	1	N/A	Passes B to Y unmodified
Bitwise AND	0x2	1	N/A	Y is bitwise AND combination of operands A and B
Bitwise OR	0x3	1	N/A	Y is bitwise OR combination of operands A and B
Bitwise XOR	0x4	1	N/A	Y is bitwise XOR combination of operands A and B
Addition	0x5	1	$A + B > 255$	Y is the sum of operands A and B
Subtraction	0x6	1	$A + (\bar{B} + 1) > 255$	Y is the sum of A and the 2's complement inverse of B
Multiplication	0x7	9	$A * B > 255$	Y is the product of A and B

Table 1: Table of supported operations

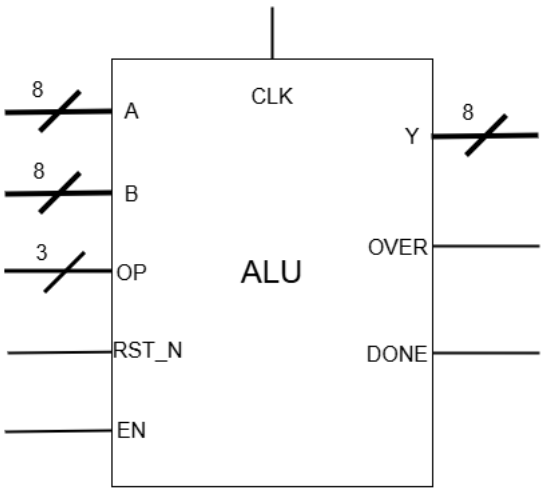


Figure 1: ALU Inputs and Outputs

Bus / Pin	Width (Bits)	Function
Input		
A	8	First 8-Bit operand - input to mathematical or logical operation.
B	8	Second 8-Bit operand - input to mathematical or logical operation.
OP	3	Opcode - selects which operation to perform.
RST_N	1	Active low asynchronous reset - puts the ALU into initial ready state.
EN	1	Enable - starts ALU when high if in ready state. Must be high for ALU to output data.
Output		
Y	8	8 Bit result - output of the ALU operation
OVER	1	Overflow flag - high value generally indicates result cannot be represented in bits of Y. Exact meaning can vary between operations.
DONE	1	Is low while operation is in progress or ALU is ready, high when operation is completed.

Table 2: ALU IO Functional Descriptions

1.1 ALU Operation

Before beginning an operation the ALU must be reset by setting RST_N low, after reset occurs the ALU is considered to be in the "ready" state. EN must be low for all rising edges of CLK while the ALU is in the ready state. Once the inputs on A, B, and OP are stable, EN can be set high to begin ALU operation. The rising edge of EN must occur when CLK is low. EN must remain high until after the next falling edge of CLK. At this point the ALU enters the "running" state. From this point on the value of EN can be changed without effecting the ALU (or the result when the ALU is done). The ALU will always output all zeros when EN is low. When DONE goes high the ALU is in the "done" state. The operation is finished, and results can be read from Y, and OVER. A timing diagram of a normal ALU operation is shown in figure 2.

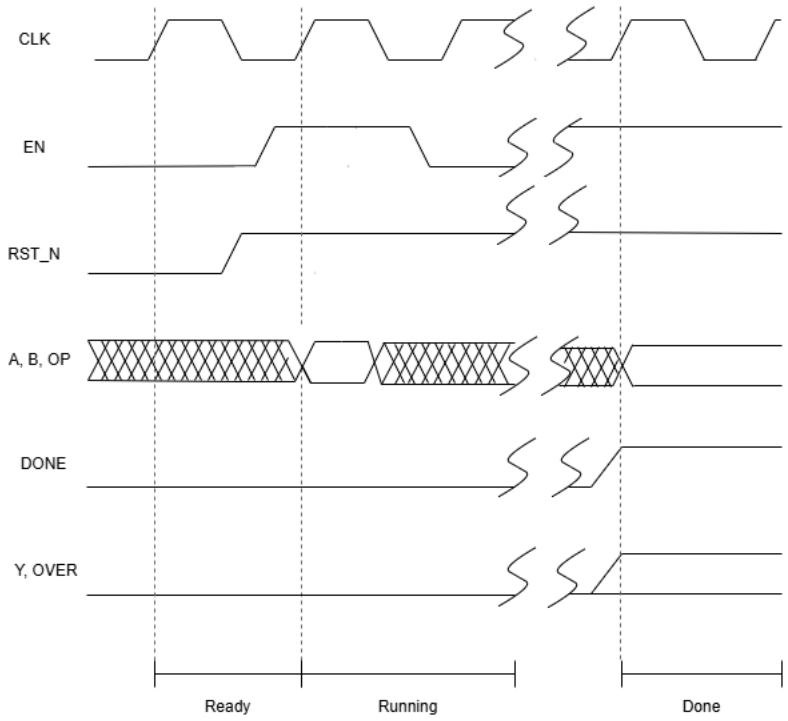


Figure 2: ALU Operation Timing Diagram

2 Design and Implementation

This ALU is designed to utilize modular operations, to allow the design to allow for easy testing during the design process, and potential rapid addition of new functions in the future. These modular operations are then linked together with integrated together with structures to allow for the selection of operations, forming the complete ALU.

2.1 Operation Modules

Each operation module accepts the inputs, and supplies the outputs shown in table 3. All outputs of the modules are routed through tristate buffers - the outputs of an operation are floating unless its enable pin is high.

There are two primary types of operation modules utilized in this ALU, combinational logic operations, and sequential logic operations. For combinational logic operations, the CLK and RST_N inputs are not needed, and DONE is always high when the module is enabled. A diagram of such an operation module is shown in figure 3. Sequential logic operations utilize all inputs to perform operations that require more than one clock cycle. The only such operation in this ALU is multiplication. A diagram of this type of module is shown in 4.

The following sections will include a breif description of the implementation of each ALU operation, and a listing of its SystemVerilog code.

2.1.1 A Pass

A Passthrough passes the 8-bit operand A to the output Y without modification. In the case enable is low, all outputs float to Z.

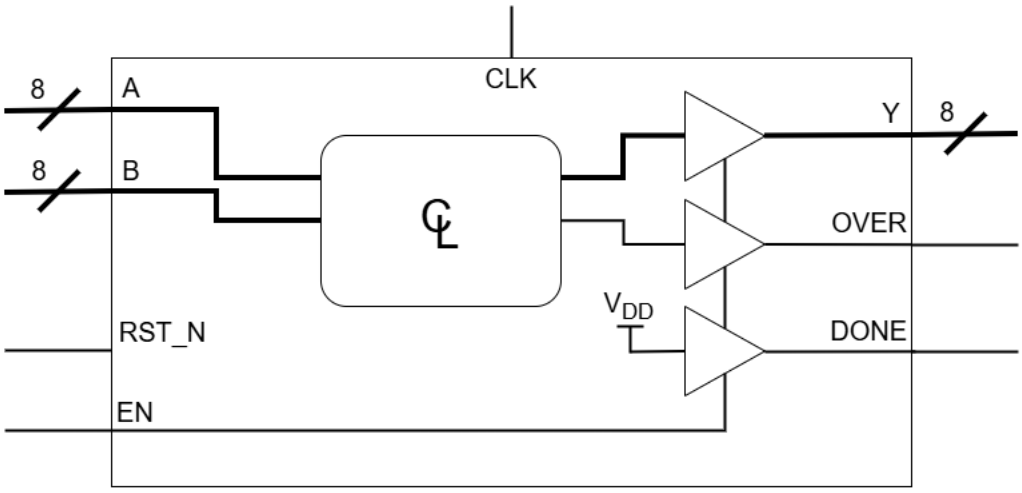


Figure 3: Schematic of a Combinational Operation Module

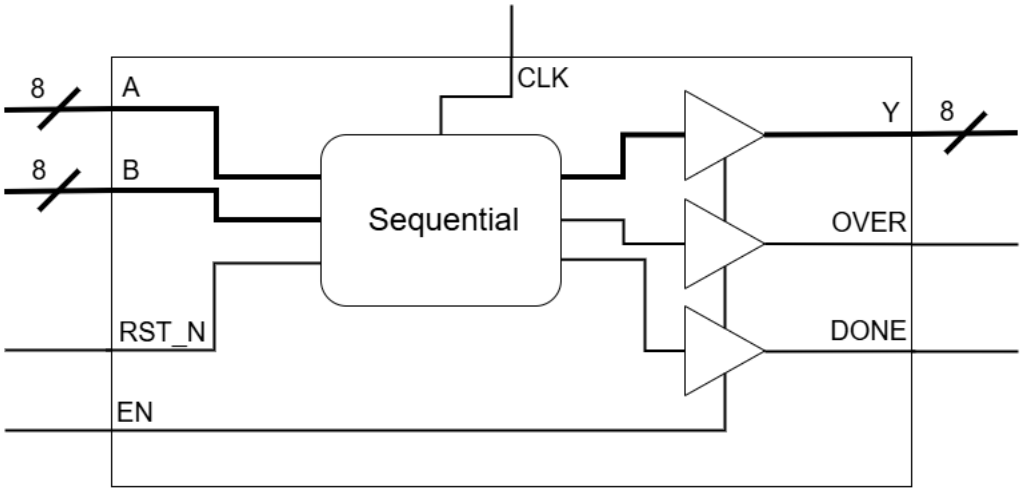


Figure 4: Schematic of a Sequential Operation Module

Listing: a_pass.sv

```
/*
Project: 8-Bit ALU
Author: John O'Connor
Sources: N/A

Module for A passthrough
*/

module A_pass(
        input logic [7:0] A,
        input logic enable,
        output logic [7:0] Y,
        output logic done,
        output logic overflow

);
```

Bus / Pin	Width (Bits)	Function
Input		
A	8	First 8-Bit operand - input to operation.
B	8	Second 8-Bit operand - input to operation.
RST_N	1	Active low asynchronous reset - resets the module to inial ready state.
EN	1	Enable - enables the module. The module will only output data when this pin is high
Output		
Y	8	8 Bit result - output of the operation
OVER	1	Overflow flag - high value generally indicates result cannot be represented in bits of Y. Exact meaning can vary between operations.
DONE	1	Is low while operation is in progress or is ready, high when operation is completed.

Table 3: Operation Module IO Descriptions

```
17
18          assign Y = enable ? A : 8'bz; //Assigns Y when enable is high
19          assign done = enable ? 1'b1 : 1'bz;  // Done is high when enable is high
20          assign overflow = enable ? 1'b0 : 1'bz;  // No overflow occurs for passthrough
21
22
23
24  endmodule
```

2.1.2 B Pass

B Passthrough passes the 8-bit operand B to the output Y without modification. In the case enable is low, all outputs float to Z.

Listing: b_pass.sv

```
1   /*
2   Project: 8-Bit ALU
3   Author: John O'Connor
4   Sources: N/A
5
6   Module for B passthrough
7   */
8
9   module B_pass(
10          input logic [7:0] B,
11          input logic enable,
```

```
        output logic [7:0] Y,
        output logic done,
        output logic overflow
);

        assign Y = enable ? B : 8'bz;  //Assigns Y when enable is high
        assign done = enable ? 1'b1 : 1'bz;  // Done is high when enable is high
    assign overflow = enable ? 1'b0 : 1'bz;  // No overflow occurs for passthrough


endmodule
```

2.1.3 Bitwise AND

Bitwise AND performs the logical AND operation on each corresponding bit of the operands A and B, setting the result of this to Y. A bit in Y is 1 if and only if the corresponding bits and A and B are booth 1. When enable is low, all outputs float.

Listing: bw_and.sv

```
/*
Project: 8-Bit ALU
Author: Max Fabian
Sources: N/A

Module for running the Bitwise AND in the ALU
*/
module Bitwise_AND(
    input logic [7:0] A, B, //8-bit Operand inputs
    input logic enable,     //Enable - sets output of operator to floating
    output logic [7:0] Y,   //8-bit Output
    output logic done,      //Done Flag
    output logic overflow   //Overflow Flag
);

    //Assigns Y to the output of Bitwise AND when enable is high
    assign Y = enable ? (A & B)  : 8'bz;

    //Simple Functions like this one have simple done and overflow flags
    assign done     = enable ? 1'b1     : 1'bz;
    assign overflow = enable ? 1'b0     : 1'bz;

endmodule
```

2.1.4 Bitwise OR

Bitwise OR performs the logical OR operation on each corresponding bit of the operands A and B, setting the result of this to Y. A bit in Y is 1 if either or both corresponding bits in A and B are 1. When enable is low, all outputs float.

Listing: bw_or.sv

```
/*
Project: 8-Bit ALU
Author: Max Fabian
Sources: N/A

Module for running the Bitwise OR in the ALU
*/
module Bitwise_OR (
    input logic [7:0] A, B, //8-bit Operand inputs
    input logic enable,     //Enable - sets output of operator to floating
    output logic [7:0] Y,   //8-bit Output
    output logic done,      //Done Flag
    output logic overflow   //Overflow Flag
);

    //Assigns Y to the output of Bitwise OR when enable is high
    assign Y = enable ? (A | B) : 8'bz;

    //Simple Functions like this one have simple done and overflow flags
    assign done     = enable ? 1'b1:1'bz;
    assign overflow = enable ? 1'b0:1'bz;

endmodule
```

2.1.5 Bitwise XOR

Bitwise XOR performs the logical XOR operation on each corresponding bit of the operands A and B, setting the result of this to Y. A bit in Y is 1 if either the of the corresponding bits in A or B is 1 but both. When enable is low, all outputs float.

Listing: bw_xor.sv

```
/*
Project: 8-Bit ALU
Author: John O'Connor
Sources: N/A

Module for Bitwise XOR.
*/

module Bitwise_XOR (
        input logic [7:0] A, B,
        input logic enable,
        output logic [7:0] Y,
        output logic done,
        output logic overflow
);


        assign Y = enable ? (A ^ B) : 8'bz;  // Assigns Y when enable is high, otherwise Y is
        ↪  floating
```

```
        assign done = enable ? 1'b1 : 1'bz;  // Done is high when enable is high
    assign overflow = enable ? 1'b0 : 1'bz;  // No overflow occurs with bitwise XOR

endmodule
```

2.1.6 Addition

Addition performs unsigned addition on two 8-bit operands A and B, setting the result of this to Y. In the case where the result of A + B exceeds 255, overflow will be set to 1, reprsenting the most significant bit. When enable is low, all outputs float.

Listing: addition.sv

```
/*
Project: 8-Bit ALU
Author: Max Fabian
Sources: N/A

Module for running the adder in the ALU
*/
module Addition(
    input logic [7:0] A, B, //8-bit operand inputs
    input logic enable,     //Enable - sets output of operator to floating
    output logic [7:0] Y,   //8-bit output
    output logic overflow,  //Overflow Flag
    output logic done       //Done Flag
);
    //Assigns overflow as 9th bit to Y and takes adder when enable is high
    assign {overflow, Y} = enable ? (A + B) : 9'bz;

    //Done flag on enable high
    assign done = enable ? 1'b1:1'bz;

endmodule
```

2.1.7 Subtraction

Subtraction performs unsigned subtraction on two 8-bit operands A and B, setting the result of this to Y. This implementation uses the two's compliment of B and computes subtraction by adding A to the two's compliment of B. When enable is low, all outputs float.

Listing: subtraction.sv

```
/*
Project: 8-Bit ALU
Author: Max Fabian
Sources: N/A

Module for running the Subtractor in the ALU
```

```
*/
module Subtraction (
    input logic [7:0] A, B, //8-bit Operand inputs
    input logic enable,     //Enable - sets output of operator to floating
    output logic [7:0] Y,   //8-bit Output
    output logic done,      //Done Flag
    output logic overflow   //Overflow Flag
);

    //Assigns Y to the output of a subtractor when enable is high
    assign Y = enable ? (A - B) : 8'bz;

    //Simple Functions like this one have simple done and overflow flags
    assign done     = enable ? 1'b1:1'bz;
    assign overflow = enable ? 1'b0:1'bz;

endmodule
```

2.1.8 Multiplication

Description here

Listing: mult.sv

```
/*
Project: 8-Bit ALU
Author: Max Fabian
Sources:
    *https://en.wikipedia.org/wiki/Binary_multiplier#Single-cycle_multiplier
    ↪ *https://stackoverflow.com/questions/63655634/8-bit-sequential-multiplier-using-add-and-shift

Sequential Multiplier, functions similarly to doing long multiplication by hand
*/
module Mult_8bit (
    input logic [7:0] A, B,//8-bit Operand inputs
    input logic clock,
    input logic reset_n,
    input logic enable,//Enable - sets output of operator to floating
    output logic [7:0] Y, // 8-bit Output
    output logic done, //Done Flag
    output logic overflow //Overflow Flag
);

//Registers for helping track the current sequence in the multiplication
logic [15:0] result;
logic [7:0]  cout;
logic [15:0]  multiplicand;
logic [3:0]  i;

    //Needs to trigger only when enable is on or when reseting
    always @(posedge (clock && enable) or negedge reset_n or negedge(enable)) begin
        //Resets all the registers and the done flag to initial states
```

```
29          if(~reset_n) begin
30              done            <= 1'b0;
31              result          <= 0;
32              multiplicand    <= A;
33              i               <= 0;
34              cout            <= 0;
35          end
36          //Sets the outputs to floating when the enable is off
37          if(~enable)begin
38              Y       <= 8'bz;
39              done    <= 1'bz;
40              overflow <= 1'bz;
41          end else begin
42              //For loop created using a counter and comparitor
43              //Functionally same as:
44              //  for(int i = 0; i < 8; i++)
45              //  counts from 0 to 7 inclusively
46              if(i < 8) begin
47                  //Performed on every iteration so the current
48                  // sequence in multiplication is correct
49                  multiplicand <= multiplicand << 1; //Logical Left Shift
50                                                     //I.E. multiplied by 2
51
52                  //Reads each bit of B and increases the result by the
53                  // current left shifted multiplicand
54                  if(B[i] == 1) begin
55                      result <= result + multiplicand;
56                  end
57              //Iterand counting by one to complete for loop functionality
58              i <= i + 1;
59
60              //Once the loop has finished the result can be thrown to
61              // the outputs, and the done flag can be raised so the ALU
62              // knows the operation has concluded.
63              end else begin
64                  {cout, Y}   = result;
65                  //Since the output of a multiplier is 2x number of
66                  // input bits and our output is staying at 8 bits
67                  // the overflow flag needs to be raised whenever
68                  // the result exceeds 8 bits
69
70                  //Done set twice here to hopefully combat race conditions
71                  if(cout > 0) begin
72                      overflow <= 1;
73                  end else begin
74                      overflow <= 0;
75                  end
76                  //Raising the done flag should always be the last
77                  // thing it does
78                  done        <= 1'b1;
79              end
80          end
81      end
82
83  endmodule
84
```

2.2 ALU Utility Modules

Three SystemVerilog modules were developed to be used within ALU operations or the ALU itself. These include a variable width asynchronously resetabble register with enable, a 3 bit multiplexer with enable, and a variable width counter. The Implementation details and SystemVerilog listings for these modules are provided in the subsequent sections.

2.2.1 3 Bit Multiplexer

The 3-bit multiplexer takes a 3-bit opcode input and converts it to an 8-bit, one-hot signal used to enable a given operation module. When enable is low, all outputs are set to 0.

Listing: multiplexer_3_en.sv

```
/*
Project: 8-Bit ALU
Author: Nolan Kessler

Implements a three bit multiplexer with enable
*/

module Multiplexer_3_en (
    input logic [2 : 0] in,
    input logic enable, //Output of multiplexer is 0 unless enable is high.
    output logic [7 : 0] out
);
    always_comb begin
        if (enable) begin
            case (in)
                3'b000: out = 8'b0000_0001;
                3'b001: out = 8'b0000_0010;
                3'b010: out = 8'b0000_0100;
                3'b011: out = 8'b0000_1000;
                3'b100: out = 8'b0001_0000;
                3'b101: out = 8'b0010_0000;
                3'b110: out = 8'b0100_0000;
                3'b111: out = 8'b1000_0000;
            endcase
        end
        else begin
            out = 4'b0;
        end
    end
endmodule
```

2.2.2 Variable Width Register

The variable width register is a configurable register, with a default width of 1, used to store the value of the in signal. If enable is high, the input "in" is copied to the output on the rising clock edge. Then, when enable is low, the register holds the value of out. If reset goes low, the output is immediately set to 0.

Listing: register_en_rstn.sv

```
/*
Project: 8-Bit ALU
Author: Nolan Kessler
Sources: Based on my register implementation from Lab 5

Implements a parameterized width register with enable and active low reset.
*/

module Register_en_rstn #(parameter WIDTH = 1) (
    input logic clk,
    input logic enable, //Enable - copy in to out on rising edge of clk only when high
    input logic rst_n, //Active low reset - set out to zero on falling edge
    input logic [WIDTH - 1 : 0] in,
    output logic [WIDTH - 1 : 0] out
);

    always_ff @(posedge clk, negedge rst_n) begin

        if (rst_n == 0) begin
            out <= 0;
        end

        else begin
            if (enable != 0) begin
                out <= in;
            end
        end

    end

endmodule
```

2.2.3 Counter

The counter is an N width counting module that increments by the input addBy. On the rising clock edge, the counter increments itself

Listing: counter.sv

```
module CounterNbit #(parameter N = 3)(
    input logic clock,
    input logic reset_n, enable_n,
```

```
    input logic [N:0] addBy,
    output logic [N:0] count
);

logic [N:0] count_next;

always_comb begin //@(posedge clock || reset_n) begin
    if (reset_n == 1'b0) begin
        count_next <= count + addBy;
    end else begin
        count_next <= 0;
    end
end

Register_en_rstn #(.WIDTH(N+1)) RegNBit (
   .clk(clock),
   .rst_n(reset_n), .enable(enable_n),
   .in(count_next),
   .out(count)
);

endmodule

```

3 Design Verification

4 Hardware Validation